

Diagrammes UML

Diagramme de cas d'utilisation & diagramme de classes

Module de gestion des bénévoles et des événements

Extension du site web vitrine de Terra Sana ASBL

Document complémentaire au cahier des charges (déjà transmis). Il rassemble les deux diagrammes demandés ainsi qu'une analyse détaillée, en particulier pour le diagramme de classes.

Encadreur scolaire : Marie-Christine Namur

Maître de stage : Didier Seraye

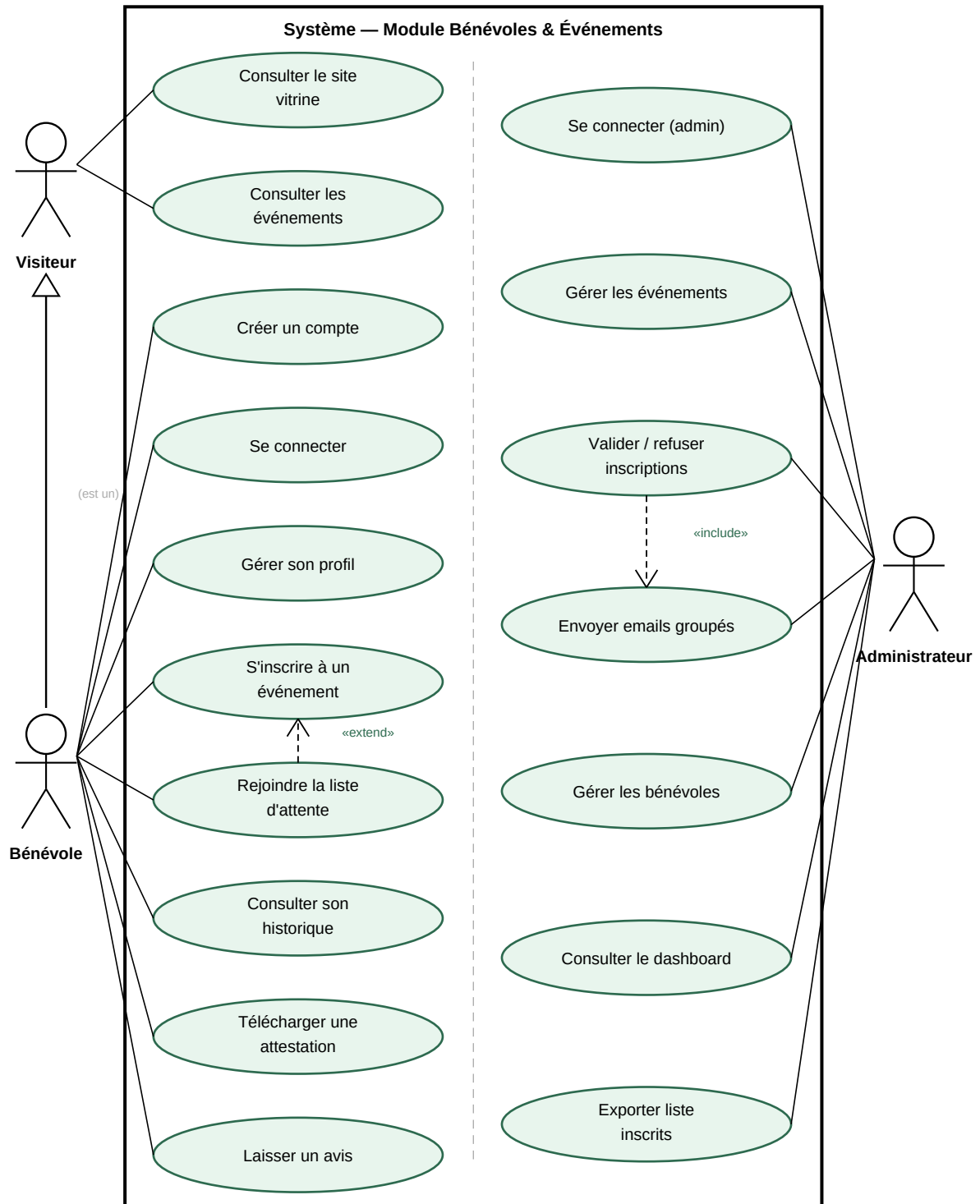
Travail présenté par Youndjeu Tchouapi Alain

EAFC Uccle

2025-2026

1. Diagramme de cas d'utilisation

Le diagramme ci-dessous décrit les interactions entre les acteurs et le module. Il distingue le périmètre côté bénévole du périmètre d'administration, tout en restant dans une seule frontière système puisqu'il s'agit d'une même application.



1.1 Les acteurs

Trois acteurs interagissent avec le module :

- **Visiteur** — internaute non authentifié. Il peut consulter le site vitrine et la liste des événements, puis créer un compte pour aller plus loin.
- **Bénévole** — visiteur qui s'est authentifié. Le lien de **généralisation** (flèche à triangle creux) indique qu'un bénévole *est un* visiteur : il hérite de toutes ses possibilités et y ajoute les siennes (inscription aux événements, profil, historique, attestations, avis).
- **Administrateur** — membre de Terra Sana qui gère le module depuis l'espace d'administration (événements, inscriptions, communication, tableau de bord).

1.2 Les relations «include» et «extend»

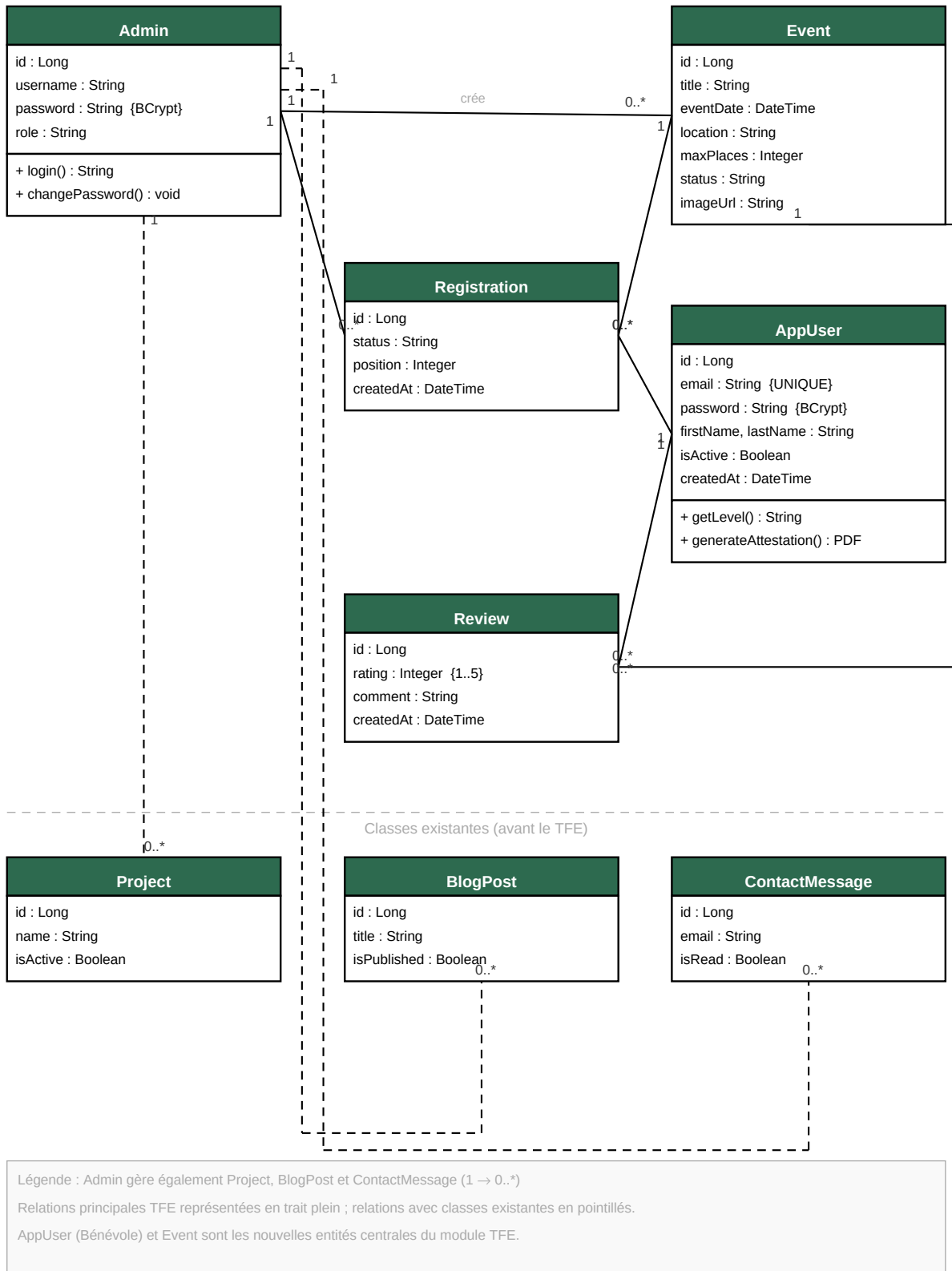
Deux relations de dépendance enrichissent le modèle et traduisent des règles métier concrètes :

- **«extend»** — « Rejoindre la liste d'attente » **étend** « S'inscrire à un événement ». C'est un comportement *optionnel*, déclenché uniquement sous condition : lorsque l'événement a atteint son nombre maximum de places, le bénévole est basculé en liste d'attente au lieu d'être inscrit directement.
- **«include»** — « Valider / refuser les inscriptions » **inclut** « Envoyer des emails ». L'envoi d'une notification au bénévole concerné fait *systématiquement* partie du traitement : il n'y a pas de validation ou de refus sans email de confirmation. La relation « include » exprime précisément ce comportement obligatoire et réutilisable.

Rappel de notation : trait plein + triangle creux = généralisation ; flèche en pointillés + pointe ouverte = dépendance «include» / «extend».

2. Diagramme de classes

Le diagramme présente le modèle de données du module. Les cinq classes du haut sont créées dans le cadre du TFE ; les trois classes situées sous le séparateur existent déjà dans la base actuelle et sont rappelées pour montrer leur rattachement à l'administrateur.



2.1 Rôle de chaque classe

Classe	Rôle dans le module
AppUser	Représente un bénévole. Entité centrale du TFE : porte l'identité, l'authentification (email unique, mot de passe chiffré) et les méthodes métier (niveau de fidélité, génération d'attestation).
Event	Un événement organisé par Terra Sana : date, lieu, nombre de places, statut (ouvert / complet / clôturé), visuel. Seconde entité centrale du module.
Registration	Classe d' association entre AppUser et Event. Elle matérialise l'inscription et porte ses propres attributs : statut (confirmée / en attente / refusée) et position dans la file d'attente.
Review	Un avis laissé par un bénévole sur un événement auquel il a participé (note de 1 à 5 et commentaire).
Admin	Compte d'administration qui gère l'ensemble du module et les contenus existants du site.

2.2 Attributs et contraintes remarquables

- **Sécurité** — les mots de passe (AppUser, Admin) ne sont jamais stockés en clair : la contrainte {BCrypt} rappelle qu'ils sont hachés. L'email de AppUser porte la contrainte {UNIQUE} pour empêcher les doublons de compte.
- **Intégrité** — la note d'un avis est bornée par {1..5} ; le champ status d'Event et de Registration prend ses valeurs dans un ensemble fermé (énumération).
- **Méthodes métier** — `getLevel()` calcule le niveau de fidélité du bénévole à partir de son nombre de participations ; `generateAttestation()` produit l'attestation PDF. Elles sont placées dans le troisième compartiment des classes concernées.

2.3 Associations et cardinalités

Association	Card.	Lecture
AppUser — Registration	1 → 0..*	Un bénévole possède plusieurs inscriptions ; une inscription appartient à un seul bénévole.
Event — Registration	1 → 0..*	Un événement reçoit plusieurs inscriptions ; une inscription concerne un seul événement.
AppUser — Review	1 → 0..*	Un bénévole peut rédiger plusieurs avis.
Event — Review	1 → 0..*	Un événement peut recevoir plusieurs avis.
Admin — Event	1 → 0..*	Un administrateur crée et gère de nombreux événements.
Admin — Registration	1 → 0..*	L'administrateur valide ou refuse les inscriptions.

La classe **Registration** est le pivot du modèle : en la plaçant entre AppUser et Event, on transforme une relation « plusieurs à plusieurs » (un bénévole s'inscrit à plusieurs événements, un événement accueille plusieurs bénévoles) en deux relations « un à plusieurs » exploitables, tout en offrant un endroit naturel pour stocker le statut et la position en liste d'attente.

2.4 Choix de conception

- **Séparation visuelle** — les classes existantes (Project, BlogPost, ContactMessage) sont regroupées sous un séparateur en pointillés. On voit d'un coup d'œil ce qui est ajouté par le TFE et ce qui préexiste, sans réécrire l'existant.
- **Réutilisation de l'administrateur** — l'Admin déjà présent pilote aussi les nouvelles entités (liens en pointillés vers les classes existantes), ce qui évite de dupliquer un compte de gestion.
- **Cohérence avec le code** — chaque classe correspond à une entité JPA (Spring Boot) et à une table MySQL ; les attributs et types reflètent directement le schéma de la base.

Les deux diagrammes sont cohérents entre eux : chaque cas d'utilisation manipule une ou plusieurs de ces classes (« S'inscrire à un événement » crée une Registration, « Laisser un avis » crée une Review, « Gérer les événements » agit sur Event, etc.).